HTML & JavaScript Workshop

JAVASCRIPT IN ACTION

WWW.KareemAboKhater.COM

Objectives :

- JavaScript: Learning how to add interactivity to websites
- Hands-on coding: Writing code and testing in real-time.
- Mini-project: A fun final project using HTML & JavaScript.

ENVIRONMENT SETUP

WWW.KareemAboKhater.COM

HTML STRUCTURE

Open Tag

Close Tag

<h1> Hello, World! </h1>

Content

JavaScript Structure

Declaration

Assignment

semicolon

```
var name = "Ali";
```

Variable Name

Initial Value

WWW.KareemAboKhater.COM

Syntax:

Case Sensitivity

```
var test = 1;
var Test = 1;
var tEst = 1;
var TEST = 1;
var teSt = 1;
var tesT = 1;

console.log(TEST);
```

Syntax:

Case Sensitivity

White Space

```
var message = "Hello from dark side.";
console.log(message);
```

Syntax:

Case Sensitivity

White Space

Identifiers

```
var Test = "test";
var test = "test";
var TEST = "test";
var _test = "test";
var Test1 = "test";
var $test = "test";
```

JavaScript - Syntax

Syntax:

Case Sensitivity

White Space

Identifiers

Comments

```
// var message = "Hello from dark side.";

/*
  Some kind
  of
  comment
  😎
*/
```

WWW.KareemAboKhater.COM

Variables Types:

In JavaScript we have 2 main kinds of types: primitive types and object types.

Primitive types are:

Number

```
var number = 10;
var fullName = "ALi"
```

String

```
var fullName = 'Ali'
var fullName = `Ali`
```

Boolean

```
var isStudent = true;
```

And two special types: null and undefined

Variables Types:

Object types have properties and also have methods that can act on those properties.

Object

```
var greeting = "Hello World!";
console.log(greeting);
```

Method

Expressions:

Arithmetic Operators :

(+, -, *, /, %)

```
let number1 = 5, number2 = 10;

console.log(number1 - number2);
console.log(number1 * number2);
console.log(number1 / number2);
console.log(number1 % number2);
```

Expressions:

Assignment Operators:

(=, +=, -=,...)

```
var num = 10;
var num2 = 20;
var result = num += num2;
console.log(result);
```

Expressions:

Logical Operators :

(===, ==, !==, !=)

```
let number1 = 5, number2 = 10;
console.log(number1 == number2);
console.log(number1 === number2);
console.log(number1 !== number2);
```

Expressions:

Comparison Operators :

(&&)

```
let number1 = 5, number2 = 10;
console.log(number1 === number2 && number1 < number2);
```

number1	number2	Result
True	True	True
True	False	False
False	True	False
False	False	False

WWW.KareemAboKhater.COM

Expressions:

Comparison Operators :

(||)

```
let number1 = 5, number2 = 10;
console.log(number1 === number2 || number1 < number2);
```

number1	number2	Result
True	True	True
True	False	True
False	True	True
False	False	False

WWW.KareemAboKhater.COM

Expressions:

Order Operators :

- *() Grouping*
- ! (Not Operator)
- * , / , %
- + , -
- >,<, <=,>=
- === , ==
- &&
- ||

```
let num = 2 + 5 * 2;
console.log(num);
```

Conditionals - Control Flow: (IF-Else)

With the comparison operators in place, we can talk about conditionals.

Condition

```
if (true) {
  //do something
}
```

Block Of Code

Conditionals - Control Flow: (IF-Else)

```
if(true){
  console.log("True");
}else{
  console.log("False");
}
```

Conditionals - Control Flow: (Switch)

```
let fruit= "apple";

switch (fruit){
  case "apple":
    console.log("Apple is red");
    break;
    case "banana":
    console.log("Banana is yellow");
    break;
  default:
    console.log("I don't know this fruit");
    break;
}
```

Condition

Case-1

Case-2

default

Loops:

For Loop:

We use the for keyword and we pass a set of 3 instructions: the initialization, the condition, and the increment part.

Initialization

Increment

```
for (let i = 0; i <= 10; i++) {
  console.log(i);
}
```

Condition

Loops:

While Loop:

Initialization

Condition

```
let i = 0;

while (i <= 10;) {
   console.log(i);
   i++
}
```

Increment

Loops:

While Loop:

Break, Continue

```
let i = 0;

while (i <= 10;) {
   console.log(i);
   if(i === 5) break;
   i++
}
```

Break keyword !

Functions:

Declaration

Function Name

Function keyword

```
function greeting() {
  console.log(`Hello World!👋`);
}
greeting();
```

Function Body

Function Call

WWW.KareemAboKhater.COM

Functions:

Declaration

Function Name

Parameter

Function keyword

```
function greeting(name) {
  console.log(`Hello ${name}👋`);
}
```

Function Body

Function Call

```
greeting("Ahmed");
```

Argument

WWW.KareemAboKhater.COM

Functions:

Declaration

Function Name

Parameter

Function keyword

```
function greeting(name) {
  console.log(`Hello ${name}👋`);
}
```

Function Body

Function Call

```
greeting("Ahmed");
```

Argument

WWW.KareemAboKhater.COM

Functions:

Declaration

Anonymous Function

```
let greeting = function(name) {
  console.log(`Hello ${name}👋`);
}

greeting("Ahmed");
```

JavaScript - Function

Functions:

Declaration

Arrow Function:

Parameters

Arrow Syntax

Function Definition

```
let add = (a, b) => {
  return a + b;
};
```

Function Body

Function Call

```
let result = add(10, 20);
```

```
console.log(result);
```

Output

WWW.KareemAboKhater.COM

Arrays:

An array is a collection of elements.

We can initialize an empty array in these 2 different ways:

```
let numbers = [10,20,30];
let numbers = Array(10,20,30);
```

Array Keyword

Arrays:

Operations

Add

```
let fruits = ["Apple", "Banana", "Orange"];
fruits.push(5);
console.log(fruits);
["Apple", "Banana", "Orange", 5]
```

Add an element at the end

Arrays:

Operations

Add

```
let fruits = ["Apple", "Banana", "Orange"];
fruits.unshift(20);
console.log(fruits);
[20,"Apple", "Banana", "Orange"]
```

Add an element at the beginning

Arrays:

Operations

Remove

```
let fruits = ["Apple", "Banana", "Orange"];
fruits.pop();
console.log(fruits);
["Apple", "Banana"]
```

Remove an element at the end

Arrays:

Operations

Remove

```
let fruits = ["Apple", "Banana", "Orange"];
fruits.shift();
console.log(fruits);
["Banana", "Orange"]
```

Remove an element at the beginning

JavaScript - Object

Object:

Declaration

Any value that's not of a primitive type (a string, a number, a boolean, a symbol, null, or undefined) is an **object**.

```
let car = new Object;
```

new keyword

```
let car = Object.create;
```

Object keyword

```
let car = {};
```

Object literal

Object types have properties and also have methods that can act on those properties.

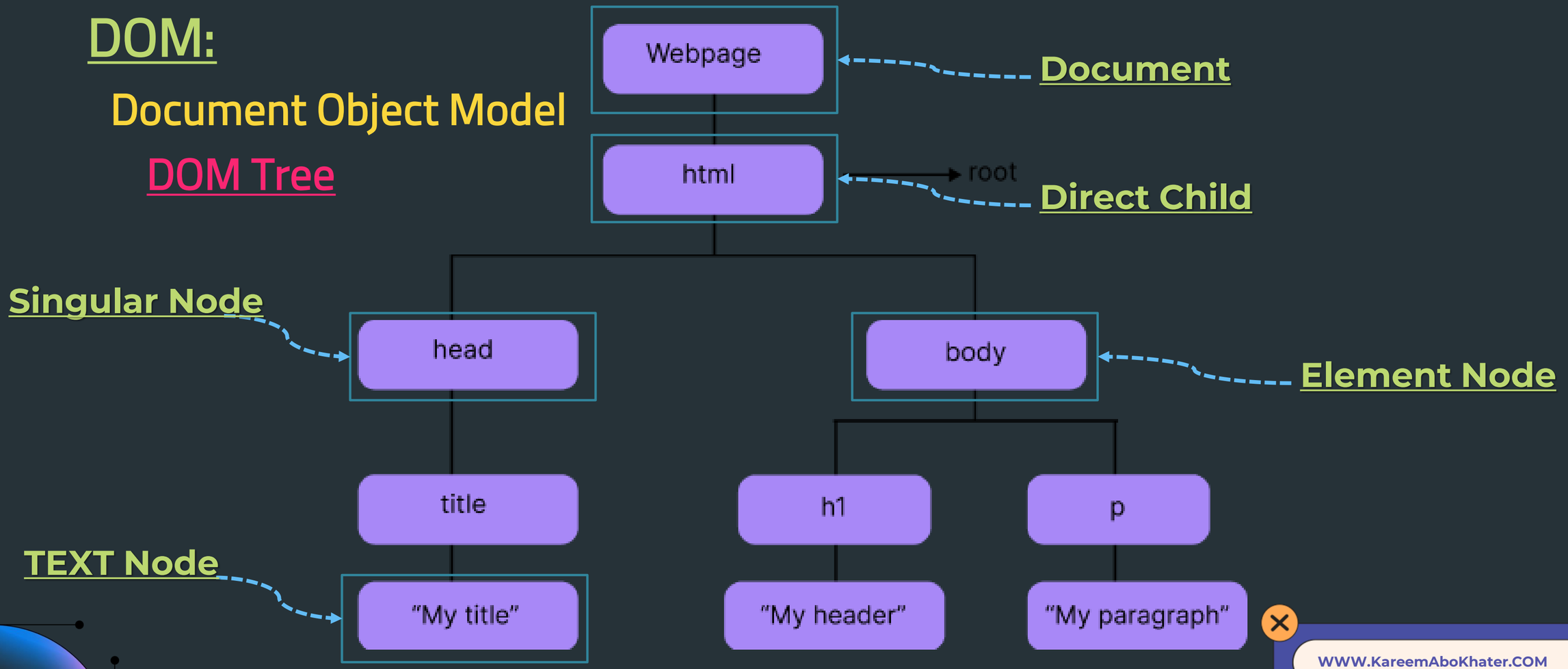
WWW.KareemAboKhater.COM

JavaScript – DOM Tree

DOM:

Document Object Model

DOM Tree



WWW.KareemAboKhater.COM

JavaScript – DOM Traversal

DOM:

Traversal allows us to navigate and manipulate elements in an HTML document using JavaScript.

DOM Traversal

```
document.querySelector(selector) → Selects the first matching element.

document.querySelectorAll(selector) →  Selects all matching elements (returns a NodeList).

document.getElementById(id) →  Selects an element by ID.

document.getElementByClassName(className) →  Selects elements by class (HTMLCollection).

document.getElementByTagsName(tagName) →  Selects elements by tagName (HTMLCollection).
```

DOM:

DOM Manipulation

`element.textContent` → Gets or sets the text content.

`element.innerHtml` → Gets or sets the HTML content.

JavaScript – DOM Traversal

DOM:

Traversal allows us to navigate and manipulate elements in an HTML document using JavaScript.

```html
<ul id="fruits">
    <li>🍎 Apple</li>
    <li>🍌 Banana</li>
    <li>🍇 Grape</li>
</ul>
```

```javascript
let list = document.getElementById("fruits");

console.log(list.firstElementChild.textContent);
console.log(list.lastElementChild.textContent);
console.log(list.children[1].textContent);
```

JavaScript – DOM Manipulation

DOM:

DOM Manipulation

```
<main id="container">
    <h1 id="blog-title">My Blog</h1>
</main>
```

```
// ADD Content
postTitle.textContent = 'DOM';
postBody.textContent = 'hello from DOM';
postDate.textContent = '16/2/2025';
```

```
// Create Element
const postTitle = document.createElement('h3');
const postDate = document.createElement('span');
const postBody = document.createElement('p');
```

```
// ADD Style
postTitle.classList.add('blog-entry');
postBody.className = 'post-body';
postDate.setAttribute('class','post-date’);
```

```
// ADD into Page
mainContainer.appendChild(postTitle);
mainContainer.appendChild(postDate);
mainContainer.appendChild(postBody);
```

DOM:

Event Listener

```html
<button id="btn" type="submit">Click Me</button>
```

```js
const btn = document.getElementById('btn');

btn.addEventListener('click',function(e){
    const target = e.target;

    alert(target.textContent)
});
```

JavaScript – DOM Manipulation

DOM:

Event Listener

Event Delegation

```html
<ul id="list">
    <li>Task 1</li>
    <li>Task 2</li>
    <li>Task 3</li>
</ul>
```

```javascript
const list = document.getElementById('list');

 list.addEventListener('click',function(e){
     if(e.target.matches('li')){
         e.target.style.textDecoration =
 'line-through';
     }
 });
```

WWW.KareemAboKhater.COM

THANK YOU!

WWW.KareemAboKhater.COM